RAZORPAY WEB PAYMENT MODULE

Client Setup Guide

How to add a Pay button to your React website, step by step

Platform: React website (Vite, Create React App, Next.js) with Node.js, Express and MySQL

Version: 1.1 **Date:** November 10, 2026

> **Note** Start in Razorpay Test Mode with fake money. Use live keys only after the "Before you go live" list at the end is complete.
>
> **Web only** This module works in web browsers. It does not work inside a native phone app (Expo Go, Android or iOS apps).

1. What you receive

You receive one folder named razorpay-payment-module-web. Only two items inside it go into your project:

MODULE FOLDER

```
razorpay-payment-module-web/ ├── razorpay-backend/          ← COPY this folder (the server
part)
├── frontend/
│   └── paymentmodule.jsx      ← COPY this file (the Pay button)
├── README.md                  ← do not copy
└── package-lock.json          ← do not copy
```

Think of your website as a shop with two parts. The front end is the counter, which is the Pay button the customer sees. The back end is the back room, which talks to Razorpay and saves every payment in MySQL. You set up both parts and connect them.

2. What you need before you start

Requirement	How to check or get it
Node.js (LTS) and npm	Run `node -v` and `npm -v` in a terminal. Both must show a version number.
MySQL server	Running on your computer or a server, with a user (such as root) that can create databases.
Razorpay account in Test Mode	Get the Key ID and Key Secret from the API Keys page of your Razorpay dashboard while Test Mode is on. Test Key IDs start with rzp_test_.
A React website	Any React project that you can run in a browser.

3. Files at a glance

This table shows every file you copy, create or edit. The steps after it give the exact commands.

Action	File or folder	Where it goes in your project
COPY	razorpay-backend/ (whole folder)	Project root, next to your src/ folder and package.json
COPY	frontend/paymentmodule.jsx	src/components/paymentmodule.jsx (create the components folder if it is missing)

CREATE	razorpay-backend/.env	Inside the razorpay-backend folder. Make it from .env.example and add your keys
EDIT	Your own server file (only if you have one)	Add one line so your server uses the payment routes
EDIT	Your page file, for example src/App.jsx	Add the payment card where you want the Pay button

Your project will look like this when you finish:

PROJECT LAYOUT

```
my-project/ ├── razorpay-backend/          ← copied
from the module
│   ├── .env                   ← created by you (keep it private)
│   ├── paymentApis.js
│   ├── server.js
│   └── database/
├── src/
│   ├── components/
│   │   └── paymentmodule.jsx  ← copied from the module
│   └── App.jsx                ← edited by you
└── package.json
```

4. Step-by-step setup

> **Tip** Open a terminal in your project's root folder (the folder that contains src/ and package.json) and keep it open for Steps 1 to 6. In the commands below, replace /path/to/ (Mac or Linux) or C:\path\to\ (Windows) with the place where you unzipped the module.

Step 1: Copy the razorpay-backend folder

Copy the whole razorpay-backend folder into your project root. You can also drag and drop it in your file explorer.

MAC / LINUX

```
cp -r /path/to/razorpay-payment-module-web/razorpay-backend ./razorpay-backend
```

WINDOWS (COMMAND PROMPT)

```
xcopy /E /I "C:\path\to\razorpay-payment-module-web\razorpay-backend" razorpay-backend
```

The folder is named razorpay-backend, not backend, so it will not overwrite a backend folder that you may already have.

Step 2: Install the packages

TERMINAL

```
cd razorpay-
backend npm
install cd ..
```

This installs express, razorpay, mysql2 and dotenv inside razorpay-backend. The last command brings you back to the project root.

Step 3: Create the .env file

Make a copy of .env.example named .env:

MAC / LINUX

```
cp razorpay-backend/.env.example razorpay-backend/.env
```

WINDOWS (COMMAND PROMPT)

```
copy razorpay-backend\.env.example razorpay-backend\.env
```

Open razorpay-backend/.env in any text editor and fill in your values:

Setting	What to enter
RAZORPAY_KEY_ID	Your Test Key ID, which starts with rzp_test_
Setting	**What to enter**
RAZORPAY_KEY_SECRET	Your Test Key Secret. Keep it private
DB_HOST	localhost (or your MySQL server address)
DB_USER	Your MySQL user, for example root
DB_PASS	Your MySQL password
DB_NAME	payments_db (any name, but it must match Step 4)
ADMIN_API_KEY	A long random string that only you know. It unlocks payment history and refunds (see the note below)
CORS_ORIGIN	* while testing. In production use your website address, for example https://myshop.com

Security The Key Secret belongs on the server only. Never put it in your React code, send it to a customer or commit it to Git.

Create the admin key. Run this in a terminal and paste the output as ADMIN_API_KEY:

TERMINAL

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Keep this key private. Anyone who has it can view all payments and issue refunds. Each request to /history and /refund must send it in the header `x-admin-key`.

If your project uses Git, stop Git from saving the secret keys:

TERMINAL

```
echo razorpay-backend/.env >> .gitignore
```

Step 4: Create the database table

Make sure MySQL is running, then run this from the project root. Use your own MySQL password:

TERMINAL

```
node razorpay-backend/database/migrate.js --db "MYSQL" --user "root" --pass "your_password"
```

Wait for the message Migration done. This creates the database and a payments table. It does not touch any other tables you have.

Important If you changed DB_NAME in .env to something other than payments_db, add --name "your_db_name" to the command. The two names must match, or the server will look for a database that does not exist.

Step 5: Start the back end

Choose the case that matches your project.

Case A: you do not have a server of your own

TERMINAL

```
node razorpay-backend/server.js
```

You should see: API on http://localhost:5000. Leave this terminal window running and open a second terminal for the next steps.

Case B: you already have an Express server

Open your server file and add the last line below. It must come after app.use(express.json()). Do not run razorpay-backend/server.js, because your own server is already running.

YOUR SERVER FILE

```
const express = require('express'); const app = express();
app.use(express.json());
app.use('/api/payments', require('./razorpay-
backend/paymentApis')); // add this line
```

If your server already uses /api/payments for something else, use another path such as /api/razorpay here and use the same path in Step 7. **Check that the back end works**

Check the history route with your admin key. Replace YOUR_ADMIN_API_KEY with the value from your .env file. A working back end shows [] (an empty list). Without the key, the server replies 401 Unauthorized:

TERMINAL

```
curl -H "x-admin-key: YOUR_ADMIN_API_KEY" http://localhost:5000/api/payments/history
```

Step 6: Copy the React component

MAC / LINUX

```
mkdir -p src/components cp /path/to/razorpay-payment-module-
web/frontend/paymentmodule.jsx \
src/components/paymentmodule.jsx
```

WINDOWS (COMMAND PROMPT)

```
mkdir src\components copy "C:\path\to\razorpay-payment-module-
web\frontend\paymentmodule.jsx" ^      src\components\paymentmodule.jsx
```

If the components folder already exists, the mkdir command may show a message. You can ignore it.

Step 7: Add the payment card to your page

Open the page where you want the Pay button and add the card. Pick your file from this table:

Your project	File to edit	Import path
React with Vite or Create React App	src/App.jsx or src/App.tsx	./components/paymentmodule
Next.js	A page file that starts with "use client"	Adjust to the page location, for example ../components/paymentmodule
Any other page of your own	That page's file	Path from that file to src/components/paymentmodule

Leave files such as main.jsx, main.tsx and index.tsx as they are. They only start the app.

SRC/APP.JSX

Example for src/App.jsx. Replace every value with your own product details:

```
import { RazorpayPaymentCard } from './components/paymentmodule';
export default function App() {
 return (
  <RazorpayPaymentCard
   apiBase="http://localhost:5000/api/payments"
   title="Your product name"
   desc="A short description of the product"
   image="https://your-site.com/product.png"
   price={0}
   currency="INR"
   email="customer@example.com"
   themeColor="#2f80ed"
   buttonText="Pay now"
   onSuccess={(res) => console.log('Paid', res)}
   onFailure={(err) => console.log('Failed', err)}
  />
 );
}
```

The price is in rupees. The card turns it into paise for Razorpay by itself. Set apiBase to the full address of your back end, as above. If you used another path in Step 5 Case B, change /api/payments to that path.

> **TypeScript** If a .tsx file shows the error "could not find a declaration file" for paymentmodule.jsx, open tsconfig.json and set "allowJs": true.

Step 8: Test a payment

TERMINAL (SECOND WINDOW, PROJECT ROOT)

```
npm run dev
```

Use npm start instead if your project was made with Create React App. Then:

#	What to do
1	Open your website in the browser and click the Pay button.
2	Pay with the test card or test UPI details from Razorpay's Test Mode documentation. No real money is used.
3	Open a terminal and run the history command below. Replace YOUR_ADMIN_API_KEY with your key.
4	Your payment should be listed with the status paid.

HISTORY

```
curl -H "x-admin-key: YOUR_ADMIN_API_KEY" http://localhost:5000/api/payments/history
```

To test a refund, copy the payment_id from the history output (it starts with pay_) and run:

MAC / LINUX

```
curl -X POST http://localhost:5000/api/payments/refund \   -H "Content-Type:
application/json" \
  -H "x-admin-key: YOUR_ADMIN_API_KEY" \
  -d '{"paymentId":"pay_XXXXXXXX"}'
```

WINDOWS (COMMAND PROMPT)

```
curl -X POST http://localhost:5000/api/payments/refund ^   -H "Content-Type:
application/json" ^
  -H "x-admin-key: YOUR_ADMIN_API_KEY" ^
  -d "{\"paymentId\":\"pay_XXXXXXXX\"}"
```

5. Reference

Card options (props)

Prop	Default	What it does
apiBase	/api/payments	Address of the back end routes
title	Payment	Product name shown on the card and in the Razorpay window
desc	empty	Short description
image	none	Image address shown at the top of the card
price	0	Price in rupees
currency	INR	Currency code
email	empty	Pre-fills the email in the Razorpay window and saves it with the order
themeColor	#2f80ed	Color of the button and the Razorpay window
buttonText	Pay ₹... with Razorpay	Custom button label
onSuccess	none	Function called after the payment is verified
onFailure	none	Function called when the order, payment or verification fails
keyId	from server	Not needed. The server sends the correct key

Back end routes

Method	Route	Purpose
POST	/create-order	Creates a Razorpay order and saves it as created
POST	/verify	Checks the payment signature and the paid amount, then marks the order paid or failed
GET	/history	Lists saved payments, newest first. Admin key required (header x-admin-key)
POST	/refund	Refunds a payment in full, or in part when you send an amount in paise. Admin key required (header x-admin-key)

6. If something goes wrong

What you see	What to do

"Network request failed" or "Failed to fetch" in the browser	The back end is not running or apiBase is wrong. Start it (Step 5) and check the address and port 5000.
"401 Unauthorized" from /history or /refund	The x-admin-key header is missing or does not match ADMIN_API_KEY in .env. Send the exact key and restart the back end after editing .env.
Migration says "Access denied"	The MySQL user or password in the command is wrong. Check them and run Step 4 again.
Server says the database does not exist	DB_NAME in .env does not match the name used in Step 4. Make them the same.
"Invalid amount" message	price must be a number greater than 0.
Razorpay window does not open	Check that the keys in .env are correct and start with rzp_test_. Restart the back end after editing .env.
Payment stays "created" in history, not "paid"	The verify call did not complete. Check that the back end is running and look for errors in its terminal window.
Payment marked "failed" with "Amount mismatch"	The amount Razorpay has for the order does not match the amount saved when the order was created. Check the price and do not change amounts between create and pay.
What you see	**What to do**
"Port 5000 is already in use"	Another program is using port 5000. Close it, or use Case B in Step 5 with your own server.
Red error when importing paymentmodule.jsx in a .tsx file	Set "allowJs": true in tsconfig.json.

7. Before you go live

The module works in Test Mode as provided. Complete these items before you accept real payments:

Item	What to do
Live keys	Put live keys in the production .env. Keep the Key Secret on the server.
HTTPS	Serve both your website and the API over HTTPS.
CORS	Set CORS_ORIGIN to your exact website address instead of *.
Admin key	Use a long random ADMIN_API_KEY that is different in every environment. Keep it out of React code and Git. /history and /refund check this key, but they do not identify individual staff members.
Restrict admin access	Block public access to /history and /refund at your server or firewall, or put them behind your own admin login. Anyone with the key can view payments and refund them.
Price on the server	The price is sent from the browser. For a real shop, look up the price on the server by product and ignore the amount from the browser.

Quick checklist

☐ Copied razorpay-backend/ into the project root and ran npm install

☐ Created razorpay-backend/.env with keys, database details and ADMIN_API_KEY

☐ Created the table with migrate.js (--name matches DB_NAME)

☐ Started the back end and saw [] at /api/payments/history with the admin key

☐ Got 401 Unauthorized at /api/payments/history without the admin key

☐ Copied paymentmodule.jsx to src/components/

☐ Added RazorpayPaymentCard to my page with the correct apiBase

☐ Made one test payment and saw it as paid in /history

☐ Finished the "Before you go live" list